Module 4 Critical Thinking - Option 2:

Laplacian Filters for Different Kernel Windows

Alexander Ricciardi

Colorado State University Global

CSC515: Foundations of Computer Vision

Dr. Bingdong Li

October 11, 2026

Module 4 Critical Thinking - Option 2: Laplacian Filters for Different Kernel Windows

Laplacian filtering can make an object's boundaries easier to see, but it also reacts to image noise. In this project, the Python program `Module4_CTA_Option2.py` (see attached Python script) compares Gaussian smoothing, Laplacian filtering, and Gaussian smoothing followed by Laplacian filtering. Each method is applied to the same photograph using 3 × 3, 5 × 5, and 7 × 7 kernels. The results are examined to determine how much background noise remains and whether the person's hair, collar, and folded arms are still recognizable; please see Appendix Figure A3 for terminal screenshots and images of the examination and results.

Preparing the Image

The program uses OpenCV to load a 250 × 166-pixel color photograph that contains noticeable fine-grained noise, particularly in the background. OpenCV reads the color channels in blue, green, and red (BGR) order. In `load_image()`, `cv2.cvtColor()` converts the photograph to grayscale, giving each pixel one brightness value instead of three color values. This allows the filters to examine changes in brightness without processing each color channel separately. The image keeps its original dimensions. The grayscale values are then converted from `uint8` to `float64`. This is important because Gaussian smoothing can produce fractional averages, while Laplacian filtering can produce positive and negative responses. Note that a response is the numerical value calculated by the Laplacian filter for each pixel. The conversion preserves those calculated values without changing the input's 0–255 brightness scale. The original color photograph remains unchanged, and all three methods use the same grayscale input.

Gaussian Filtering and Sigma

Gaussian smoothing is a low-pass filter. It reduces rapid changes in pixel brightness, including some image noise, while retaining more gradual changes. The `cv2.GaussianBlur()` function calculates a weighted average of the pixels around a center pixel. For example, a 3 × 3 kernel uses nine positions, and the pixels nearest the center receive more weight than those farther away. The weights add up to one. This averaging reduces small fluctuations, but it can also soften object boundaries because pixels on both sides of an edge contribute to the same calculation (Dey, 2025). Kernel size determines the neighborhood used for the calculation. Sigma determines how the weights decrease with distance from the center. The program uses a sigma of 1.0 pixel in both directions for all three kernel sizes. This value was selected from the appearance of the filtered photograph rather than calculated as an optimal setting. Keeping sigma fixed makes the different windows easier to compare. At image boundaries, `cv2.BORDER_REFLECT_101` supplies reflected neighboring pixels so filtering can be performed without adding an artificial dark border.

Laplacian Filtering

Unlike Gaussian smoothing, the Laplacian calculates how changes in brightness vary across neighboring pixels. A first derivative describes the rate of intensity change, while a second derivative describes how that rate changes. The Laplacian adds the horizontal and vertical second derivatives to produce a response at each pixel. Nearly uniform regions give responses close to zero. A boundary may produce a larger positive or negative response, but noise can produce similar values. A strong response therefore does not confirm that an object boundary has been found. The `apply_laplacian()` function uses `cv2.Laplacian()` with `CV_64F` to preserve negative as well as positive values. The program also adjusts the derivative coefficient gain for

each window size, using scale factors of 1/4, 1/64, and 1/1024 for the 3 × 3, 5 × 5, and 7 × 7 kernels (OpenCV, n.d.). These factors help make response magnitudes more comparable, although a larger kernel still examines a different neighborhood.

Combining the Filters

For each kernel size, `create_results()` calculates three separate results. Gaussian smoothing and the direct Laplacian both begin with the original grayscale image. For the third result, the Laplacian is applied to the Gaussian-smoothed image from that same kernel size. For example, the 5 × 5 result is not calculated from the earlier 3 × 3 result. Keeping the three windows independent makes it possible to compare the effect of each size. Code Excerpt 1 shows this part of the program. Applying Gaussian smoothing before the Laplacian follows the idea behind the Laplacian of Gaussian (LoG), although the program performs the two filters separately. Marr and Hildreth (1980) describe identifying edges through zero crossings, where the second-derivative response changes sign. This program displays filter responses but does not select those crossings to create a final edge map.

Code Excerpt 1

Independent filtering methods in create_results()

```
    # Apply to the gray_image
    for kernel_size in KERNEL_SIZES:
        # METHOD 1: Smooth the original grayscale image for this window size.
        gaussian_image = apply_gaussian(gray_image, kernel_size, sigma)
        # METHOD 2: Apply the Laplacian directly to the original pixels.
        # METHOD 3: Apply it to this row's Gaussian-smoothed pixels.
        # Keep all three arrays, including the intermediate Gaussian result.
        results[kernel_size] = {
            "gaussian": gaussian_image,
            "laplacian": apply_laplacian(gray_image, kernel_size),
            "gaussian_laplacian": apply_laplacian(gaussian_image, kernel_size),
        }
```

Note: This code snippet showcases the code used to implement the Gaussian and Laplacian filters on the grayscale image, with the Gaussian smoothing filter being applied before the Laplacian filter.

Displaying and Saving the Results

Matplotlib arranges all nine results in three rows for kernel size and three columns for filter type. The Gaussian panels display grayscale intensities on the regular 0–255 scale. For the Laplacian panels, the program takes the absolute value of each response, divides it by one maximum shared by all six derivative panels, and applies a square-root adjustment. This makes weaker responses easier to see while keeping equal magnitudes equally bright across panels. Only the displayed images are adjusted. The signed numerical results are saved separately in `filter_arrays.npz`, along with the PNG images and a five-stage terminal explanation.

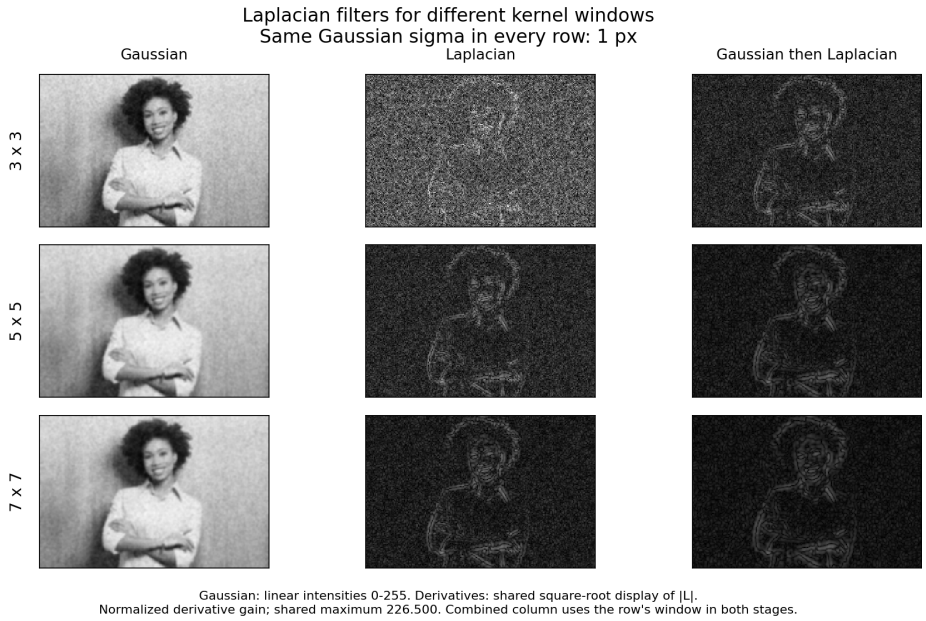
Observed Results

With the 3 × 3 Gaussian filter, some of the noise is reduced, although the background still has a grainy appearance. The 3 × 3 Laplacian highlights many small changes in brightness across the background, making the person's outline harder to distinguish. Applying Gaussian smoothing before the Laplacian reduces some of these unwanted changes. With the 5 × 5 combined filter, the hair, collar, sleeves, and folded arms are still recognizable, and there is less interference from the background. It is important to keep these features visible because their boundaries help define the person's shape. The 7 × 7 result also shows the general outline, but the Laplacian responses around the face and arms become wider, making some of the smaller details harder to distinguish; see Figure 1.

Another observation is that the 5 × 5 and 7 × 7 Gaussian results look almost identical when sigma is set to 1.0. Their mean absolute difference is only 0.134188 intensity levels, measured on the grayscale scale of 0 to 255. At this sigma setting, the additional outer positions included in the 7 × 7 kernel have very small weights and contribute little to the calculated pixel values. The Laplacian results, however, show greater differences as the kernel size increases.

Figure 1

Gaussian, Laplacian, and Gaussian-then-Laplacian filtering for three kernel sizes



Note: All Gaussian filters use sigma 1.0. The Laplacian and combined-filter images show the absolute values of the calculated responses, without indicating whether the original values were positive or negative.

Additionally, the program measures the strength of the Laplacian responses in the same area of the background to the right of the person. For this comparison, the program uses root-mean-square (RMS). The calculation involves squaring each response value, averaging the squared values, and calculating the square root of that average. Squaring the values prevents the positive and negative responses from canceling each other out. For the 5 × 5 kernel, the Laplacian by itself produces an RMS of 8.5802. When Gaussian smoothing is applied before the Laplacian, the RMS decreases to 2.9419. This reduction agrees with what can be observed in Figure 1, where fewer bright marks appear in the background after smoothing. However, the measured area also contains changes in shading that are part of the original photograph.

Therefore, a lower RMS indicates weaker Laplacian responses in that area, but it does not necessarily mean that the boundaries have been detected more accurately.

Conclusion and Limitations

For this photograph, applying Gaussian smoothing followed by the Laplacian with 5 × 5 kernels and sigma 1.0 gives the clearest result overall. The Laplacian by itself highlights small changes in background brightness, including those caused by noise, which makes the person's outline more difficult to follow. Applying Gaussian smoothing first reduces much of that background noise without losing the main boundaries around the hair, collar, and folded arms. The 7 × 7 result continues to show the larger boundaries, although some of the smaller features become harder to distinguish.

The results are consistent with the approach described by Marr and Hildreth (1980), where Gaussian smoothing is applied before calculating the second derivatives. Canny (1986) also explains that an edge detector should identify edges, locate them accurately, and avoid producing multiple responses along the same edge. However, Canny's method uses first derivatives after smoothing, rather than the Laplacian used in this program. It is also important to consider that the program calculates and displays filtering results, not completed edge maps or sharpened photographs. Since no reference image with labeled edges is available, the accuracy of the detected boundaries cannot be determined. The 5 × 5 kernel was selected based on the results obtained from this particular photograph using sigma 1.0. Different photographs or sigma settings could produce different results

References

Canny, J. (1986). A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence, PAMI-8*(6), 679–698. https://doi.org/10.1109/TPAMI.1986.4767851

Dey, S. (2025). *Image processing and computer vision masterclass with Python: Learn advanced image processing techniques, computer vision fundamentals, and applications* (2nd ed.). BPB Publications.

Marr, D., & Hildreth, E. (1980). Theory of edge detection. *Proceedings of the Royal Society of London. Series B. Biological Sciences, 207*(1167), 187–217. https://doi.org/10.1098/rspb.1980.0020

OpenCV. (n.d.). *Image filtering*. OpenCV 4.13.0 documentation. https://docs.opencv.org/4.13.0/d4/d86/group__imgproc__filter.html

Appendix

Program Output Screenshots

This section is reserved for screenshots from the program run.

Figure A1

Original Image



Note: Original image from assignment

Figure A2

Gaussian Followed by Laplacian: Kernels Comparison



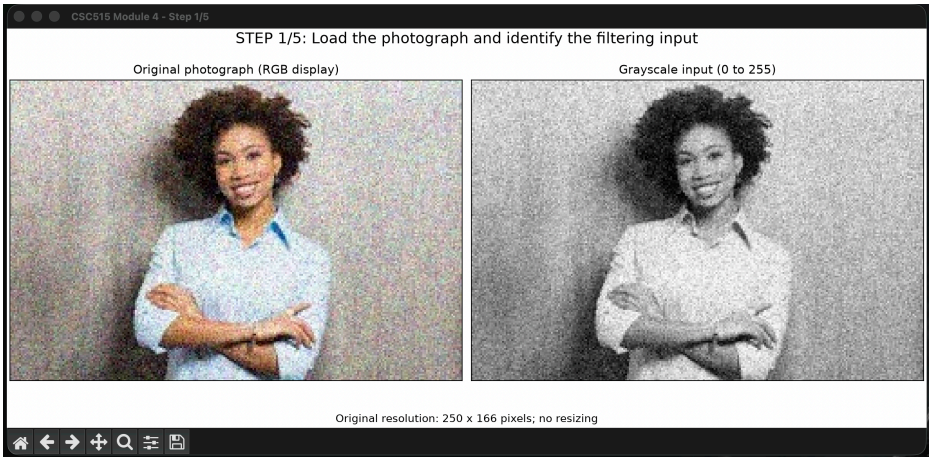
Note: Final generated images using Gaussian followed by Laplacian using 3x3, 5x5, and 7x7 kernels for comparison purposes.

Please see the next page

Figure A3

Terminal Walkthrough





```
Enter C to continue or Q to quit, then press Enter.
c
Opening step 2/5...

================================================================================
STEP 2/5: Gaussian filtering and the meaning of sigma
================================================================================
Gaussian smoothing calculates a new pixel intensity from a weighted average of nearby intensities.
It can reduce small changes caused by noise, but it can also soften details and boundaries.

This run uses sigma = 1, measured in pixels, in both directions. The same sigma is used for the 3 x
3, 5 x 5, and 7 x 7 windows, as required by the assignment. Kernel size selects how many neighboring
pixels enter the calculation. Sigma controls how much their influence decreases with distance from
the center. All the weights add up to one.

Code in apply_gaussian():

    return cv2.GaussianBlur(
        gray_image, (kernel_size, kernel_size), sigmaX=sigma, sigmaY=sigma,
        borderType=BORDER_TYPE,
    )

BORDER_TYPE is cv2.BORDER_REFLECT_101. When a filter window reaches the edge of the photograph,
OpenCV reflects nearby pixels to complete that neighborhood. This avoids adding a black border
during filtering. All three methods use the same setting.

Gaussian weight matrices for the three window sizes:

Each number is the weight assigned to one position in the window. The center receives more weight
than distant pixels. Very small values may appear in scientific notation; for example, 2e-05 means
0.00002.

    3 x 3; weight sum = 1.000000
    [[0.0751 0.1238 0.0751]
     [0.1238 0.2042 0.1238]
     [0.0751 0.1238 0.0751]]

    5 x 5; weight sum = 1.000000
    [[0.003  0.0133 0.0219 0.0133 0.003 ]
     [0.0133 0.0596 0.0983 0.0596 0.0133]
     [0.0219 0.0983 0.1621 0.0983 0.0219]
     [0.0133 0.0596 0.0983 0.0596 0.0133]
     [0.003  0.0133 0.0219 0.0133 0.003 ]]

    7 x 7; weight sum = 1.000000
    [[1.9652e-05 2.3941e-04 1.0730e-03 1.7690e-03 1.0730e-03 2.3941e-04
      1.9652e-05]
     [2.3941e-04 2.9166e-03 1.3071e-02 2.1551e-02 1.3071e-02 2.9166e-03
      2.3941e-04]
     [1.0730e-03 1.3071e-02 5.8582e-02 9.6585e-02 5.8582e-02 1.3071e-02
      1.0730e-03]
     [1.7690e-03 2.1551e-02 9.6585e-02 1.5924e-01 9.6585e-02 2.1551e-02
      1.7690e-03]
     [1.0730e-03 1.3071e-02 5.8582e-02 9.6585e-02 5.8582e-02 1.3071e-02
      1.0730e-03]
     [2.3941e-04 2.9166e-03 1.3071e-02 2.1551e-02 1.3071e-02 2.9166e-03
      2.3941e-04]
     [1.9652e-05 2.3941e-04 1.0730e-03 1.7690e-03 1.0730e-03 2.3941e-04
      1.9652e-05]]

Actual 3 x 3 neighborhood centered at [row=83, column=125]:

    [[205. 191. 167.]
     [231. 197. 175.]
     [196. 182. 185.]]

Multiply each of these nine pixel intensities by the Gaussian weight at the same position:

    [[15.398 23.654 12.544]
     [28.607 40.223 21.672]
     [14.722 22.539 13.896]]

Example: Adding the nine products gives 193.256451. This is the new center-pixel intensity.
GaussianBlur() gives 193.256451 at the same location.

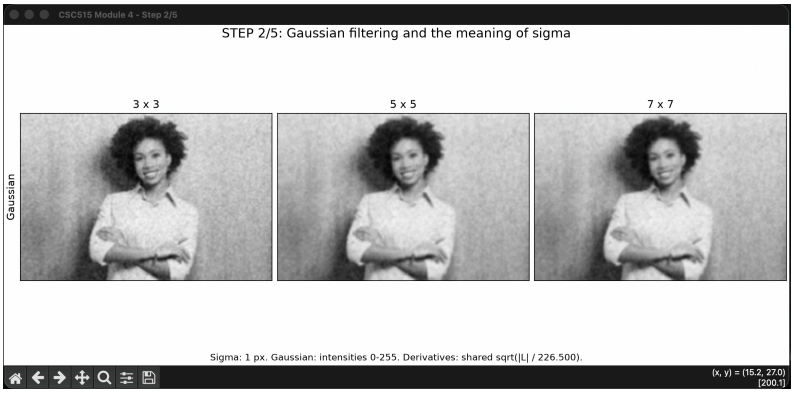
Reading the pictures:

From left to right, the panels show smoothing with 3 x 3, 5 x 5, and 7 x 7 windows. Compare how much
background texture becomes less noticeable and whether small features also become softer. With sigma
held constant, the 5 x 5 and 7 x 7 results may look very similar because the additional outer
weights contribute only a small amount.

All three panels show grayscale intensities on the same 0-255 scale: 0 is black and 255 is white.
Gaussian smoothing does not calculate a Laplacian response; that is examined in the next step.

Saved teaching view:
/Volumes/P-SSD/Omegapy/CSU-projects/CSC515-CTAs/CTA_Module-4/mod_images/diagnostics/walkthrough/stage_02.png

Enter C to continue or Q to quit, then press Enter.
```



```
Enter C to continue or Q to quit, then press Enter.
c
Opening step 3/5...

================================================================================================
STEP 3/5: Use the Laplacian to examine possible boundaries
================================================================================================
The direct Laplacian examines brightness changes without Gaussian smoothing first. It makes many
boundaries stand out, but it also responds to small changes in texture and noise.

A first derivative describes how brightness changes between nearby positions. A second derivative
describes how that rate of change changes across the image. The Laplacian adds the horizontal and
vertical second derivatives. Nearly uniform areas give values close to zero. Strong changes can give
positive or negative responses, depending on the local brightness pattern.

Code in apply_laplacian():

    return cv2.Laplacian(
        gray_image, ddepth=cv2.CV_64F, ksize=kernel_size,
        scale=laplacian_scale(kernel_size), delta=0, borderType=BORDER_TYPE,
    )

OpenCV calculates the Laplacian using second-derivative coefficients. The scale setting adjusts
their numerical gain for each window size: 1/4 for 3 x 3, 1/64 for 5 x 5, and 1/1024 for 7 x 7. A
larger window still examines a different neighborhood. CV_64F keeps negative values instead of
losing them in an 8-bit image; delta=0 adds no offset.

Direct Laplacian results before converting them for display:

    Aperture     Scale          Signed minimum     Signed maximum
    3 x 3      0.250000000       -203.5000           226.5000
    5 x 5      0.015625000        -77.5000            79.0938
    7 x 7      0.000976562        -55.8867            52.8457

The 3 x 3 Laplacian weight matrix used in this calculation:

    [[ 0.5  0.   0.5]
     [ 0.  -2.   0. ]
     [ 0.5  0.   0.5]]

The center has a negative weight and the diagonal neighbors have positive weights. These
coefficients add up to zero. If every pixel in the neighborhood has the same brightness, their
weighted sum is approximately zero. Different surrounding intensities can produce a signed response.

Laplacian calculation at [row=83, column=125]:

    [[ 102.5    0.    83.5]
     [   0.  -394.     0. ]
     [  98.     0.    92.5]]

The program uses the same nine pixels from Step 2, but multiplies them by the Laplacian coefficients
instead of Gaussian weights. The products sum to -17.500000; cv2.Laplacian() returns -17.500000. The
result keeps its positive or negative sign.

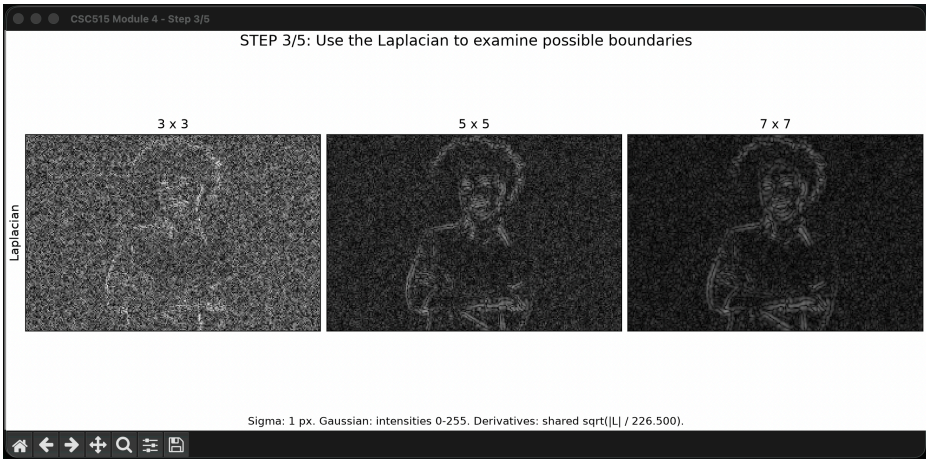
Reading the pictures:

From left to right, these are the direct 3 x 3, 5 x 5, and 7 x 7 Laplacian results. Bright areas
represent a stronger response, whether the original value was positive or negative. Compare the main
outlines with the smaller bright marks in the background. Some of those marks may come from image
noise rather than a useful feature.

The six Laplacian panels in Steps 3 and 4 use the same display scale, based on maximum absolute
response 226.500000. The full signed values remain available in the saved numerical arrays.

Saved teaching view:
/Volumes/P-SSD/Omegapy/CSU-projects/CSC515-CTAs/CTA_Module-4/mod_images/diagnostics/walkthrough/stage_03.png

Enter C to continue or Q to quit, then press Enter.
```



```
Enter C to continue or Q to quit, then press Enter.
c
Opening step 4/5...

================================================================================================
STEP 4/5: Smooth first, then apply the Laplacian
================================================================================================
This method applies Gaussian smoothing first, then calculates the Laplacian from that smoothed
image. The purpose is to reduce small brightness fluctuations before examining possible boundaries.

The program starts each kernel size with the original grayscale image. For the combined result, it
passes the Gaussian output directly to the Laplacian. Both operations use that row's 3 x 3, 5 x 5,
or 7 x 7 window, and every Gaussian uses the same sigma.

Code in create_results():

    gaussian_image = apply_gaussian(gray_image, kernel_size, sigma)
    results[kernel_size] = {
        "gaussian": gaussian_image,
        "laplacian": apply_laplacian(gray_image, kernel_size),
        "gaussian_laplacian": apply_laplacian(gaussian_image, kernel_size),
    }

How the two filtering operations are connected:

    gray_image -> Gaussian(k, sigma) -> Laplacian(k) -> signed response
    k = 3           3 x 3               3 x 3       support 5 x 5
    k = 5           5 x 5               5 x 5       support 9 x 9
    k = 7           7 x 7               7 x 7       support 13 x 13

In create_results(), the direct Laplacian uses gray_image, while the combined result uses
gaussian_image. This is the difference being tested. For an interior pixel, the support sizes above
show the total original-image neighborhood that can affect the final result after both operations.
They do not mean that a separate 9 x 9 or 13 x 13 Laplacian was applied.

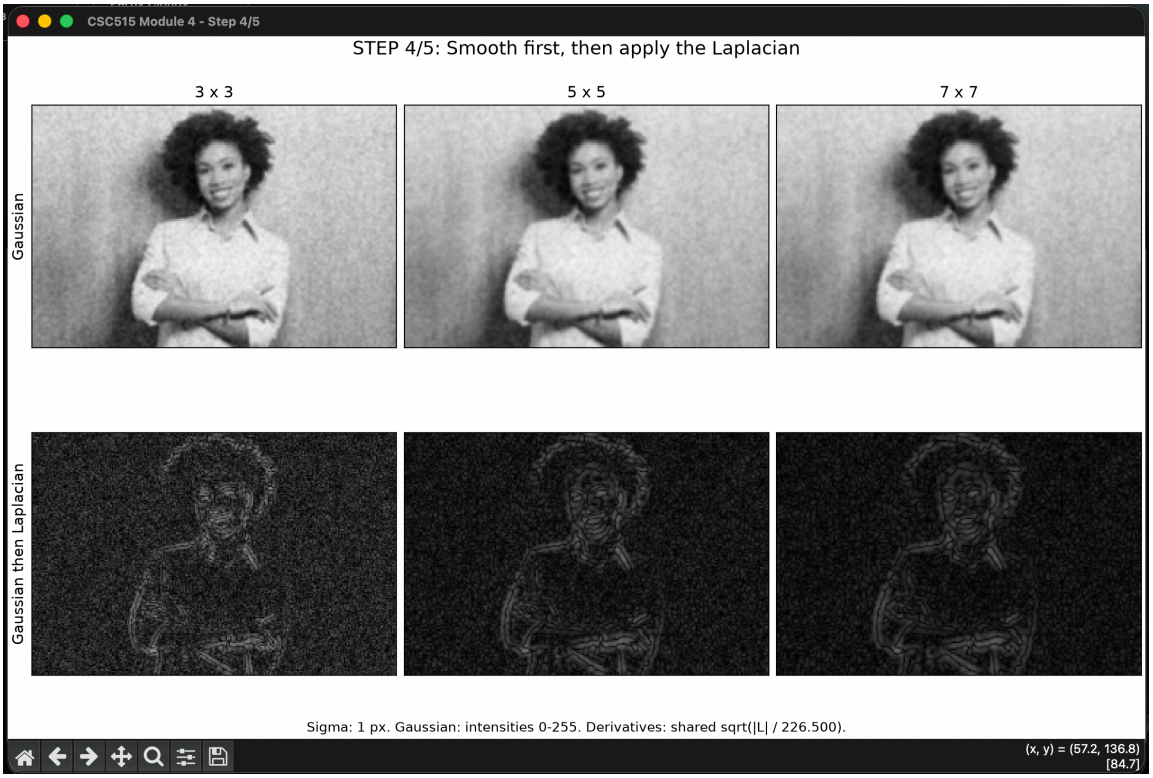
Reading the pictures:

Each column uses one window size. The upper image shows the Gaussian result, and the lower image
shows the Laplacian calculated from it. Compare the lower images with the direct Laplacian pictures
in Step 3. Look for less background clutter while checking whether the main outlines and smaller
details remain visible.

The upper images show grayscale brightness. The lower images show the strength of the Laplacian
response, so a bright area has a different meaning in each row.

Saved teaching view:
/Volumes/P-SSD/Omegapy/CSU-projects/CSC515-CTAs/CTA_Module-4/mod_images/diagnostics/walkthrough/stage_04.png

Enter C to continue or Q to quit, then press Enter.
```



```
Enter C to continue or Q to quit, then press Enter.
c
Opening step 5/5...

================================================================================
STEP 5/5: Compare the results and check what was saved
================================================================================
The final 3 x 3 figure contains the nine filtering results required by the assignment. It brings the
three methods and three window sizes together for a direct visual comparison.

Each row uses one window size: 3 x 3, 5 x 5, or 7 x 7. Each column shows Gaussian, direct Laplacian,
or Gaussian followed by Laplacian. Read across to compare the methods, then down to compare window
sizes while the Gaussian sigma stays fixed.

How the Laplacian responses are displayed in grayscale:

    display = (abs(response) / shared_maximum) ** 0.5

The six Laplacian views share a maximum absolute response of 226.500000. abs() lets positive and
negative responses appear as brightness; the square root makes weaker responses easier to see. The
same response strength therefore looks equally bright in every Laplacian panel. This changes the
display only. Gaussian panels still use the regular 0–255 grayscale intensity scale.

Example: The average absolute pixel difference between the 5 x 5 and 7 x 7 Gaussian results is
0.134188 intensity levels. A smaller number means the two smoothed images are more alike at
corresponding pixel positions.

For the supplied photograph, the program measures a small area in the right background, using rows
20:140 and columns 195:235. RMS (root mean square) summarizes the strength of the Laplacian values:
square them, average them, and take the square root. Compare the direct and combined columns to see
how smoothing changes the response in that area. The selected area contains shading as well as
noise.

Background response measurements:

    Aperture      Laplacian RMS      Gaussian + Laplacian RMS
    3 x 3             38.5412                     7.5658
    5 x 5              8.5802                     2.9419
    7 x 7              4.3050                     1.9716

Reading the pictures:

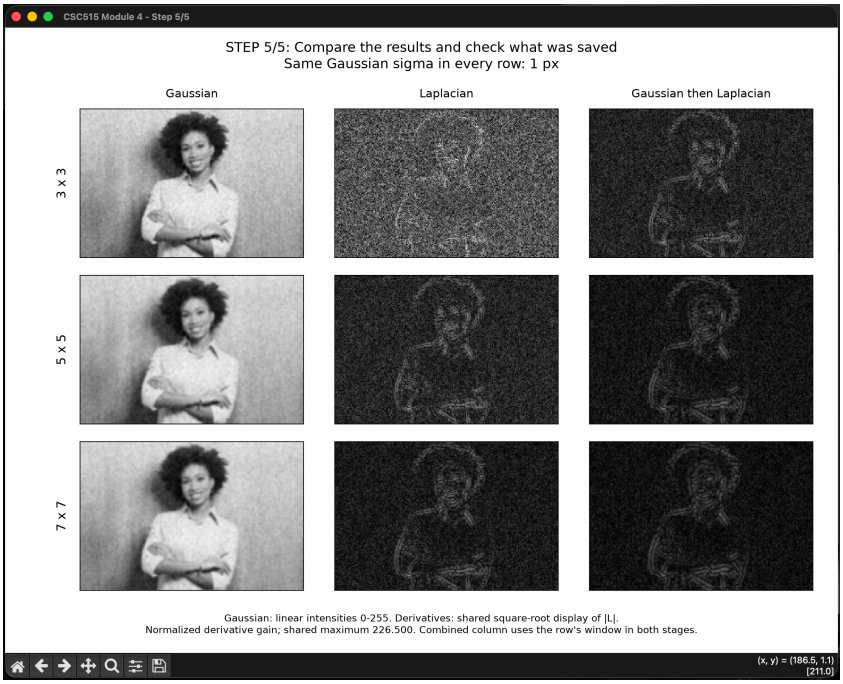
For the supplied photograph, I prefer Gaussian followed by Laplacian using a 5 x 5 window and sigma
1.0. The direct 3 x 3 result contains more small background responses. Smoothing first reduces some
of those responses while keeping the hair outline, shirt collar, and crossed arms recognizable. The
7 x 7 result can soften narrower details, including parts of the hair and face. Those features
matter because they help describe the subject's shape.

Gaussian smoothing reduces small intensity variations but also softens details. The direct Laplacian
makes changes stand out, including noise. For a noisy photograph, Gaussian followed by Laplacian can
give a useful balance: fewer background responses while keeping the larger outlines visible. The
amount of smoothing still affects fine detail.

Output folder: /Volumes/P-SSD/Omegapy/CSU-projects/CSC515-CTAs/CTA_Module-4/mod_images
The program saves the original/grayscale reference, the labeled 3 x 3 comparison, and nine
individual filtered PNGs. filter_arrays.npz holds the original signed filter values, and
terminal_output.txt contains a copy of these five explanations.

Saved teaching view:
/Volumes/P-SSD/Omegapy/CSU-projects/CSC515-CTAs/CTA_Module-4/mod_images/diagnostics/walkthrough/stage_05.png

Enter Q to quit, then press Enter.
```





Note: The figure showcases the terminal outputs and the images generated using the Matplotlib library.